

LAPORAN PRAKTIKUM 9
Rancang Bangun Sistem Monitoring dan
Prediksi Kepadatan Kendaraan Berbasis
Big Data Framework

Untuk memenuhi tugas mata kuliah Teknologi Big Data

Dosen Pengampu: Bapak Muhayat, S.Ag., M.IT.



Oleh:

Siti Magfiratun Warahmah

Nim: 230104040059

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS DAKWAH DAN ILMU KOMUNIKASI
UNIVERSITAS ISLAM NEGERI ANTASARI BANJARMASIN
TAHUN 2026

1. Tujuan Praktikum

Tujuan dari praktikum ini adalah:

- a) Mengintegrasikan Data Processing Engine (PySpark) dengan Visualization Layer (Streamlit)
- b) Memahami pentingnya penggunaan format data Parquet untuk efisiensi penyimpanan Big Data
- c) Menangani kendala Path Management (Absolute dan Relative Path) pada lingkungan Linux Server/WSL
- d) Mengimplementasikan model Machine Learning sederhana untuk prediksi deret waktu (time-series)

2. Konsep Dasar

2.1 Big Data dan Apache Spark

Big Data merupakan data dalam jumlah besar yang memerlukan teknologi khusus untuk pengolahan. Pada praktikum ini digunakan Apache Spark sebagai engine untuk memproses data kendaraan secara cepat dan efisien dengan konsep batch processing.

2.2 Konsep Utama yang Digunakan

Beberapa konsep utama dalam praktikum ini meliputi:

- a) Apache Spark untuk pengolahan data skala besar
- b) Parquet sebagai format penyimpanan data yang efisien
- c) Linear Regression untuk prediksi jumlah kendaraan
- d) Absolute Path untuk memastikan lokasi file stabil di Linux/WSL

2.3 Pengolahan Data

Data kendaraan yang berisi waktu, lokasi, dan jumlah kendaraan diolah menggunakan Spark. Proses ini meliputi perhitungan total kendaraan per lokasi, pengelompokan data berdasarkan waktu, serta pengambilan data jam untuk kebutuhan Machine Learning.

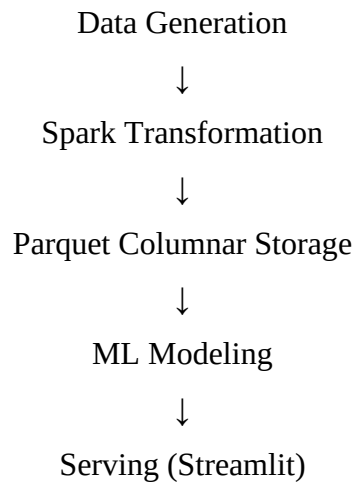
2.4 Penyimpanan dan Visualisasi

Data hasil pengolahan disimpan dalam format Parquet agar lebih cepat dibaca. Selanjutnya, data ditampilkan melalui dashboard Streamlit dalam bentuk grafik dan informasi interaktif.

2.5 Machine Learning dan Pipeline

Model Linear Regression digunakan untuk memprediksi jumlah kendaraan berdasarkan

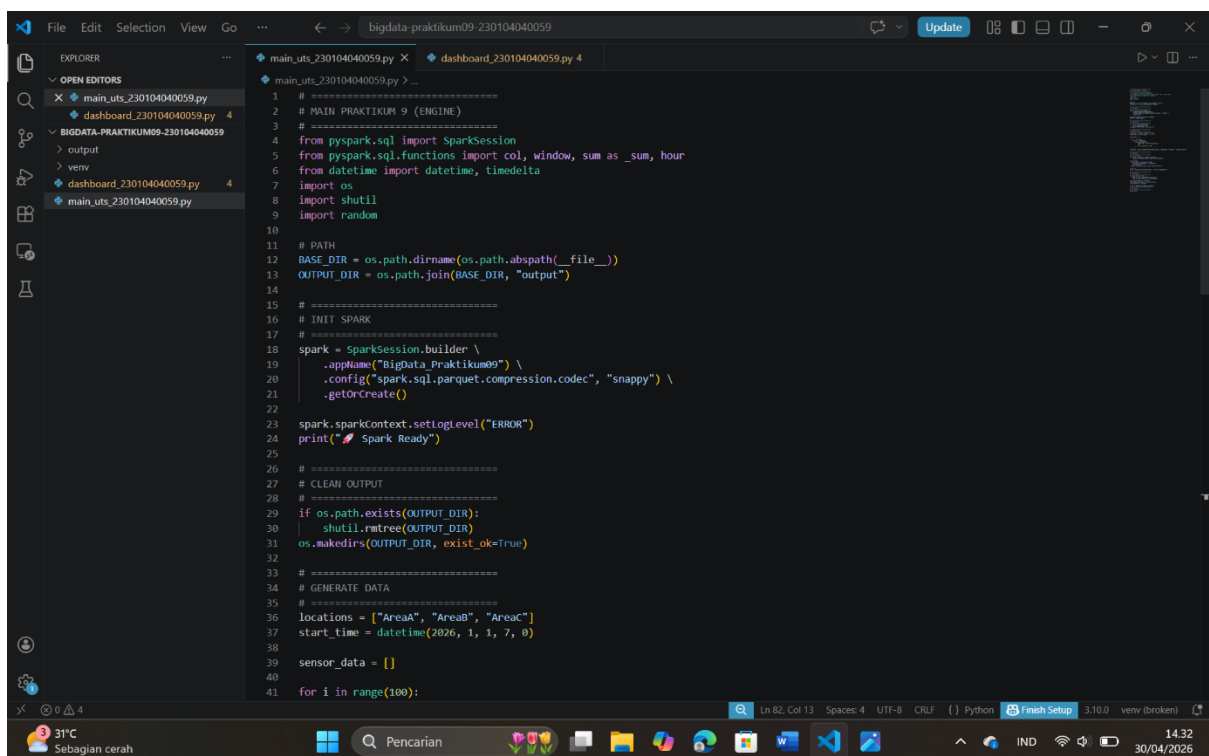
waktu. Sistem ini berjalan dalam satu alur (pipeline), yaitu:



3. Dokumentasi Implementasi Praktikum

Hasil implementasi praktikum ditunjukkan melalui beberapa dokumentasi berikut.

Screenshot Script Engine (main_uts_230104040059.py)



```
1  # =====
2  # MAIN PRAKTIKUM 9 (ENGINE)
3  # =====
4  from pyspark.sql import SparkSession
5  from pyspark.sql.functions import col, window, sum as _sum, hour
6  from datetime import datetime, timedelta
7  import os
8  import shutil
9  import random
10
11 # PATH
12 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
13 OUTPUT_DIR = os.path.join(BASE_DIR, "output")
14
15 # =====
16 # INIT SPARK
17 # =====
18 spark = SparkSession.builder \
19     .appName("BigData Praktikum09") \
20     .config("spark.sql.parquet.compression.codec", "snappy") \
21     .getOrCreate()
22
23 spark.sparkContext.setLogLevel("ERROR")
24 print("🔥 Spark Ready")
25
26 # =====
27 # CLEAN OUTPUT
28 # =====
29 if os.path.exists(OUTPUT_DIR):
30     shutil.rmtree(OUTPUT_DIR)
31 os.makedirs(OUTPUT_DIR, exist_ok=True)
32
33 # =====
34 # GENERATE DATA
35 # =====
36 locations = ["AreaA", "AreaB", "AreaC"]
37 start_time = datetime(2026, 1, 1, 7, 0)
38
39 sensor_data = []
40
41 for i in range(100):
```

```
File Edit Selection View Go ... bigdata-praktikum09-230104040059 Update
EXPLORER
main_uts_230104040059.py
dashboard_230104040059.py 4
BIGDATA-PRAKTIKUM09-230104040059
output
venv
main_uts_230104040059.py
main_uts_230104040059.py

main_uts_230104040059.py > ...
42 for loc in locations:
43     sensor_data.append((
44         start_time + timedelta(minutes=i),
45         loc,
46         random.randint(10, 100)
47     ))
48
49 sensor_df = spark.createDataFrame(sensor_data, ["timestamp", "location", "vehicle_count"])
50
51 # =====
52 # PROCESSING
53 # =====
54 # Total kendaraan
55 traffic_df = sensor_df.groupBy("location") \
56     .agg(sum("vehicle_count").alias("total_vehicle"))
57
58 # Tren waktu
59 traffic_time_df = sensor_df.groupBy(
60     window(col("timestamp"), "10 minutes"),
61     col("location")
62 ).agg(sum("vehicle_count").alias("total_vehicle"))
63
64 # Data ML
65 ml_df = sensor_df.withColumn("hour", hour(col("timestamp")))
66
67 # =====
68 # SAVE PARQUET
69 # =====
70 def save_data(df, name):
71     path = os.path.join(OUTPUT_DIR, name)
72     df.write.mode("overwrite").parquet(path)
73
74 save_data(traffic_df, "traffic")
75 save_data(traffic_time_df, "traffic_time")
76 save_data(ml_df, "ml_data")
77
78 print("SEMU DATA BERHASIL DISIMPAN")
79
80 # STOP SPARK
81 # =====
82 spark.stop()
```

Screenshot Script Dashboard (dashboard_230104040059.py)

```
File Edit Selection View Go ... bigdata-praktikum09-230104040059 Update
EXPLORER
main_uts_230104040059.py
dashboard_230104040059.py 4
BIGDATA-PRAKTIKUM09-230104040059
output
venv
dashboard_230104040059.py 4
main_uts_230104040059.py

dashboard_230104040059.py > ...
1 # =====
2 # DASHBOARD PRAKTIKUM 9
3 # =====
4 import streamlit as st
5 from pyspark.sql import SparkSession
6 import plotly.express as px
7 import pandas as pd
8 from sklearn.linear_model import LinearRegression
9 import os
10
11 # PATH
12 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
13 OUTPUT_DIR = os.path.join(BASE_DIR, "output")
14
15 st.set_page_config(page_title="Traffic Dashboard", layout="wide")
16 st.title("Smart City AI Traffic Dashboard")
17
18 # =====
19 # INIT SPARK
20 # =====
21 @st.cache_resource
22 def get_spark():
23     return SparkSession.builder.appName("Dashboard").getOrCreate()
24
25 spark = get_spark()
26
27 # =====
28 # LOAD DATA
29 # =====
30 def load_parquet(name):
31     path = os.path.join(OUTPUT_DIR, name)
32     if not os.path.exists(path):
33         st.error(f"Folder {name} tidak ditemukan. Jalankan engine dulu!")
34         st.stop()
35     return spark.read.parquet(path).toPandas()
36
37 df = load_parquet("traffic")
38 pdf_time = load_parquet("traffic_time")
39 pdf_ml = load_parquet("ml_data")
40
41 # =====
```

The screenshot shows a Visual Studio Code editor window with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'bigdata-praktikum09-230104040059' with files 'main_uts_230104040059.py' and 'dashboard_230104040059.py'. The code editor displays the 'dashboard_230104040059.py' file, which contains a Python script for a dashboard. The script includes a filter for locations, a plot of total vehicle count by location, and a linear regression model for predicting vehicle count based on time of day.

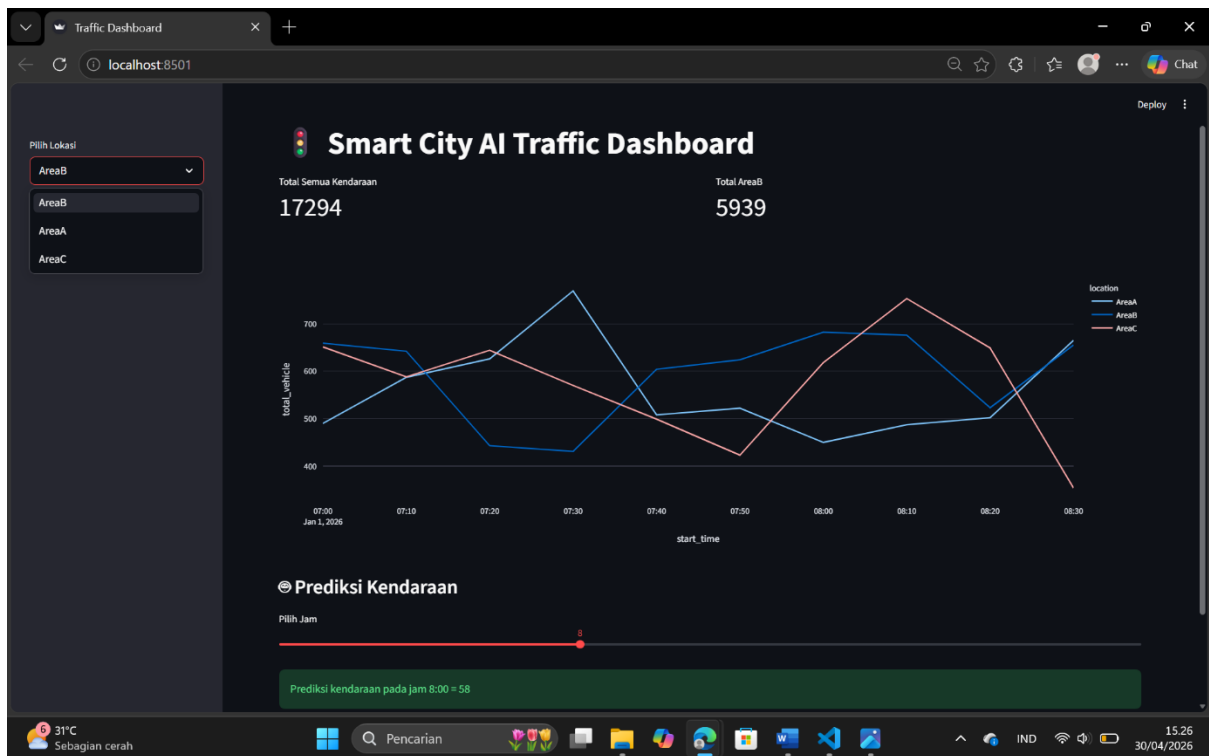
```
42 # FILTER
43 # =====
44 locations = df["location"].unique()
45 selected = st.sidebar.selectbox("Pilih lokasi", locations)
46
47 filtered = df[df["location"] == selected]
48
49 # =====
50 # KPI
51 # =====
52 col1, col2 = st.columns(2)
53
54 with col1:
55     st.metric("Total Semua Kendaraan", int(df["total_vehicle"].sum()))
56 with col2:
57     st.metric(f"Total {selected}", int(filtered["total_vehicle"].sum()))
58
59 # =====
60 # GRAFIK
61 # =====
62 pdf_time["start_time"] = pdf_time["window"].apply(lambda x: x[0] if isinstance(x, tuple) else x.start)
63
64 fig = px.line(pdf_time, x="start_time", y="total_vehicle", color="location")
65 st.plotly_chart(fig, use_container_width=True)
66
67 # =====
68 # AI PREDICTION
69 # =====
70 st.subheader("🔮 Prediksi Kendaraan")
71
72 x = pdf_ml[["hour"]]
73 y = pdf_ml[["vehicle_count"]]
74
75 model = LinearRegression()
76 model.fit(x, y)
77
78 hour_input = st.slider("Pilih Jam", 0, 23, 12)
79
80 pred = model.predict([[hour_input]])
81
82 st.success(f"Prediksi kendaraan pada jam {hour_input}:00 = {int(pred[0])}")
```

Screenshot Bukti Eksekusi Program (Engine)

The screenshot shows a Visual Studio Code editor window with a terminal panel at the bottom. The terminal displays the output of a Python script execution. The output includes a warning about the hostname, a message about the SparkUI service, and a confirmation that the data was successfully saved to a file named 'ml_data'.

```
rahmah_230104040059@mgfwr:~/bigdata-praktikum09-230104040059$ source venv/bin/activate
(venv) rahmah_230104040059@mgfwr:~/bigdata-praktikum09-230104040059$ python main_uts_230104040059.py
26/04/30 14:45:51 WARN Utils: Your hostname, mgfwr resolves to a loopback address: 127.0.1.1; using 10.255.255.254 instead
(on interface lo)
26/04/30 14:45:51 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/04/30 14:45:52 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classe
s where applicable
26/04/30 14:45:54 WARN Utils: Service 'SparkUI' could not bind on port 4040. Attempting port 4041.
Spark Ready
SEMUA DATA BERHASIL DISIMPAN
(venv) rahmah_230104040059@mgfwr:~/bigdata-praktikum09-230104040059$ ls output
ml_data traffic traffic_time
(venv) rahmah_230104040059@mgfwr:~/bigdata-praktikum09-230104040059$
```

Screenshot Visualisasi Dashboard Streamlit



4. Hasil dan Pembahasan

Berdasarkan hasil praktikum, sistem pengolahan data kendaraan menggunakan Apache Spark berhasil dijalankan dengan baik. Data simulasi kendaraan diolah untuk menghasilkan agregasi berdasarkan lokasi dan waktu, kemudian disimpan dalam format Parquet sebagai media penyimpanan yang efisien. Data tersebut selanjutnya divisualisasikan menggunakan dashboard Streamlit dalam bentuk informasi interaktif. Pada dashboard ditampilkan total seluruh kendaraan sebesar 17.294 serta total kendaraan pada lokasi yang dipilih (AreaB) sebesar 5.939.

Selain itu, grafik menunjukkan tren perubahan jumlah kendaraan pada AreaA, AreaB, dan AreaC dari waktu ke waktu. Sistem juga dilengkapi dengan fitur prediksi menggunakan metode Linear Regression. Berdasarkan hasil pada dashboard, jumlah kendaraan pada pukul 08.00 diperkirakan sebesar 58 kendaraan. Hal ini menunjukkan bahwa model dapat digunakan untuk memperkirakan kondisi lalu lintas di waktu mendatang.

4.1 Analisis Jam Tersibuk

Berdasarkan grafik pada dashboard, terlihat bahwa jumlah kendaraan mengalami fluktuasi pada setiap interval waktu. Kepadatan kendaraan mulai meningkat pada pagi hari dan mencapai puncaknya pada sekitar pukul 08.00 hingga 08.10.

Pada rentang waktu tersebut, beberapa area menunjukkan jumlah kendaraan yang tinggi secara bersamaan, terutama pada AreaB dan AreaC. Hal ini mengindikasikan bahwa waktu tersebut merupakan jam tersibuk (peak hour) dalam data yang dianalisis.

Selain itu, pemilihan waktu prediksi pada pukul 08.00 menunjukkan konsistensi dengan hasil analisis, dimana waktu tersebut termasuk dalam periode dengan tingkat kepadatan kendaraan tertinggi. Dengan demikian, dapat disimpulkan bahwa pukul 08.00–08.10 merupakan waktu dengan tingkat kepadatan kendaraan tertinggi.

5. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, sistem pengolahan dan visualisasi data kendaraan berbasis Big Data berhasil dibangun menggunakan Apache Spark, Parquet, dan Streamlit. Sistem mampu mengolah data secara efisien, menampilkan dashboard interaktif, serta melakukan prediksi jumlah kendaraan menggunakan Linear Regression. Hasil analisis menunjukkan bahwa puncak kepadatan kendaraan terjadi pada rentang waktu pukul 08.00 hingga 08.10, sehingga sistem ini dapat menjadi dasar dalam pengembangan monitoring lalu lintas berbasis Smart City.

6. Repository GitHub

Seluruh source code dan file yang digunakan dalam praktikum ini disimpan dalam repository GitHub yang dapat diakses melalui link berikut:

<https://github.com/Siti-Magfiratun-Warahmah/Big-Data-Praktikum09-Siti-Magfiratun-Warahmah-230104040059>